



Universidade Federal do Ceará
Campus Quixadá

QXD0011 - Fundamentos de Banco de Dados
Prof. Francisco Victor da Silva Pinheiro

Trabalho Final

Entrega 4 – Visões, Controle de Acesso e Aplicação Final

Quixadá-CE
2026

1. Integrantes da Dupla

Matrícula	Nome	E-mail
578795	ANTONIO MARCELO DE MENEZES SILVA	mnzes.marcelo@gmail.com
578984	JOAO VITOR DOURADO FREIRE SANTOS	joaovitor20062006@gmail.com
578168	ARTUR DE SOUSA LIMA	artursousa092@gmail.com
582635	ANTONIO KAUÁ ALVES DO NASCIMENTO	kaua.alves@alu.ufc.br
582110	GUILHERME WARLEY BRITO FARIAS	warrley@alu.ufc.br

2. Resumo da Proposta

Nessa entrega, o foco foi tirar o banco de dados do papel, deixar ele mais seguro e prepará-lo para que o sistema possa funcionar de forma real. Para isso, a gente trabalhou em três partes. Primeiro, **segurança**, pois em um banco de dados real não pode haver senhas com mais de uma pessoa podendo ter acesso. Então, criamos dois perfis, um usuário **admin**, que pode realizar alterações caso necessário, e um usuário de **leitura**. Esse segundo usuário só consegue enxergar os dados; ele não consegue deletar ou alterar qualquer coisa, o que evita falhas ou acidentes. Segundo, criamos duas **views**, que funcionam como "atalhos" para consultas que seriam gigantescas.

- A primeira é focada no **financeiro**: ela junta quatro tabelas diferentes para dar um relatório mastigado de quantas passagens da viação vendeu e qual foi o faturamento real delas.
- A segunda resolve a **logística**: ela traduz o código do banco nos nomes reais das cidades e respectivas rodoviárias, mostrando o itinerário completo de onde o ônibus sai e para onde vai, tudo em ordem de horário.

Também foi desenvolvida a **aplicação**. Criamos uma API que se conecta diretamente a esse banco. Na prática, quando o sistema precisa mostrar os caminhos dos ônibus ou o relatório de vendas na tela, ele usa aquele usuário restrito e puxa os dados direto das Views que a gente programou.

3. Visão no Banco de Dados

- **Descrição da Visão Criada**

A visão **vw_resumo_financeiro_viação** realiza o cruzamento entre as tabelas de viação, ônibus, viagem e passagem filtrando para considerar apenas as compras com o status "confirmada", agregando os resultados por nome da viação.

O propósito é calcular de forma rápida o volume de vendas efetivas. Ela extrai duas métricas principais: a quantidade total de passagens vendidas e a receita financeira

bruta gerada por cada empresa. A utilidade no sistema é gerar relatórios financeiros, permitindo visualizar o faturamento sem precisar escrever queries repetidas.

A visão **vw_itinerario_trechos** é uma visão operacional e apresentativa, abstraindo os múltiplos relacionamentos da tabela trecho, realizando joins com as tabelas rodoviária e cidade.

O propósito é mapear os segmentos das viagens de forma legível, entregando as cidades e rodoviárias de partida e chegada, além dos horários e custos. A utilidade é mostrar claramente de onde o ônibus sai e para onde vai. fornece as informações dos bilhetes, facilita a ordenação dos embarques, ordenando pelo horário e data.

- **Comando SQL da Visão**

Visão 1: Resumo de Receita e Vendas por Viação

```
CREATE OR REPLACE VIEW vw_resumo_financeiro_viacao AS
SELECT
    v.nome AS nome_viacao,
    COUNT(p.id_passagem) AS total_passagens_vendidas,
    SUM(p.valor_total) AS receita_total
FROM viacao v
JOIN onibus o ON v.id_viacao = o.id_viacao
JOIN viagem vg ON o.id_onibus = vg.id_onibus
JOIN passagem p ON vg.id_viagem = p.id_viagem
WHERE p.status = 'Confirmada'
GROUP BY v.nome;
```

Visão 2: Itinerário Detalhado dos Trechos

```
CREATE OR REPLACE VIEW vw_itinerario_trechos AS
SELECT
    t.id_trecho,
    c_origem.nome AS cidade_origem,
    r_origem.nome AS rodoviaria_origem,
```

```

c_destino.nome AS cidade_destino,
r_destino.nome AS rodoviaria_destino,
t.data_hora_saida,
t.data_hora_chegada,
t.preco_trecho
FROM trecho t
JOIN rodoviaria r_origem ON t.id_origem = r_origem.id_rodoviaria
JOIN cidade c_origem ON r_origem.id_cidade = c_origem.id_cidade
JOIN rodoviaria r_destino ON t.id_destino = r_destino.id_rodoviaria
JOIN cidade c_destino ON r_destino.id_cidade = c_destino.id_cidade
ORDER BY t.data_hora_saida;

```

4. Criação de Usuários e Controle de Acesso

- ***Usuários Criados***

Foram criados dois usuários: o usuário de leitura e um usuário administrador. O usuário administrador possui privilégios de leitura e escrita, permitindo acesso aos recursos do banco de dados, já o usuário de leitura possui permissões de leitura, como SELECT.

- ***Comandos SQL Utilizados***

```
-- 1. Criação dos usuários
```

```
CREATE USER admin_user WITH PASSWORD 'senha_admin123';
```

```
CREATE USER leitura_user WITH PASSWORD 'senha_leitura123';
```

```
-- 2. Concedendo permissões totais para o Admin (Leitura e Escrita)
```

```
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO
admin_user;
```

```
GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA public
TO admin_user;
```

-- Permite que o admin_user crie novas estruturas (Views, Tabelas, etc.) no schema public

```
GRANT CREATE ON SCHEMA public TO admin_user;
```

-- 3. Concedendo permissão de leitura para o usuário restrito

```
GRANT USAGE ON SCHEMA public TO leitura_user;
```

```
GRANT SELECT ON ALL TABLES IN SCHEMA public TO leitura_user;
```

-- 4. REVOKE Garantindo restrição explícita

```
REVOKE INSERT, UPDATE, DELETE, TRUNCATE ON ALL TABLES  
IN SCHEMA public FROM leitura_user;
```

-- 5. Garantir que tabelas criadas no futuro respeitem essas regras

```
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT ALL ON  
TABLES TO admin_user;
```

```
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT SELECT  
ON TABLES TO leitura_user;
```

5. Aplicação Desenvolvida

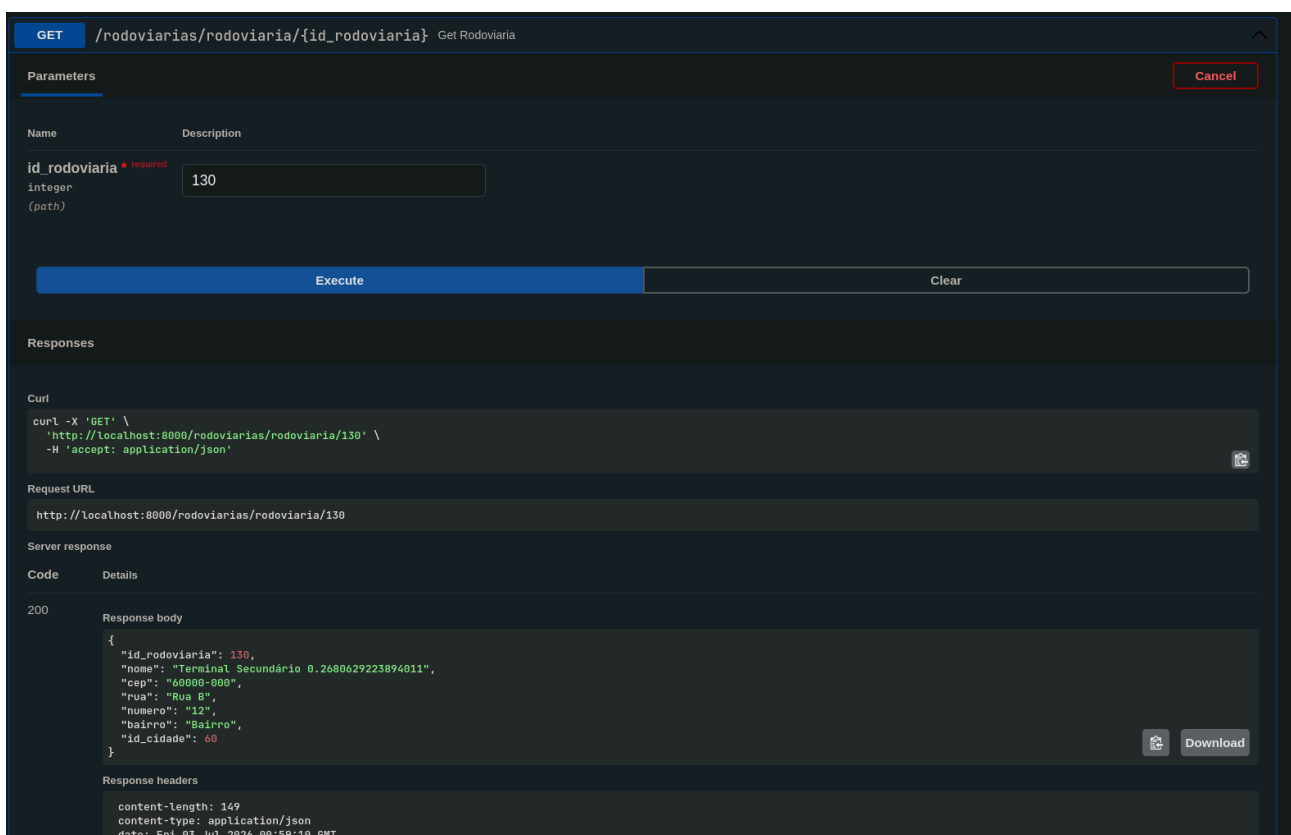
- **Linguagem/Ferramenta Utilizada**

- A linguagem utilizada foi **Python 3.11**.
- O framework utilizado foi o **FastAPI** para criação das rotas e gerenciamento da API.
- O driver do banco de dados utilizado foi **Psycopg2** que serve como ponte para de comunicação e também para executar os comandos SQL no PostgreSQL.
- Para a validação dos dados foi usado o **Pydantic** para definir os esquemas (basemodel) de entrada e saída por meio das classes Rodoviaria e RodoviariaUpdate.
- O banco de dados relacional usado foi o **PostgreSQL**.

- **Funcionalidades Implementadas**

- **Cadastro de Rodoviária (POST /rodoviaria):** Insere uma nova rodoviária no banco de dados vinculando-a a uma cidade (id_cidade). A operação conta com controle de transação (commit/rollback) para garantir a consistência caso ocorra algum erro durante o processo.
- **Listagem Geral de Rodoviárias (GET /rodoviaras):** Recupera todos os registros da tabela rodoviaria via SELECT, mapeando os resultados em uma lista de objetos validados pelo Pydantic para o retorno da API.
- **Consulta por ID (GET /rodoviaria/{id_rodoviaria}):** Permite buscar uma única rodoviária específica informando o seu identificador numérico. Caso o registro não exista no banco, a API retorna um erro customizado 404 (Not Found).
- **Remoção de Rodoviária (DELETE /rodoviaria/{id_rodoviaria}):** Remove de forma direta o registro correspondente ao ID enviado utilizando a cláusula DELETE FROM ... WHERE id_rodoviaria=%s.
- **Atualização Parcial (PATCH /rodoviaria/{id_rodoviaria}):** Permite modificar dinamicamente apenas os campos enviados pelo cliente (como nome, CEP, rua, etc.). O sistema utiliza o método model_dump(exclude_unset=True) do Pydantic para capturar o que foi alterado e monta a query UPDATE em tempo de execução, tratando erros com rollback seguro.

- Prints ou Capturas de Tela da Aplicação



GET /rodoviar/rodoviar Listar Rodoviar

Parameters

No parameters

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8080/rodoviar/rodoviar' \
  -H 'accept: application/json'
```

Request URL

http://localhost:8080/rodoviar/rodoviar

Server response

Code

Details

200

Response body

```
{
  "numero": "481",
  "bairro": "Centro",
  "id_cidade": 1
},
{
  "id_rodoviaria": 2,
  "nome": "Terminal Rodoviário de Juazeiro do Norte 2",
  "cep": "38438-000",
  "rua": "Av. Central",
  "numero": "788",
  "bairro": "Centro",
  "id_cidade": 2
},
{
  "id_rodoviaria": 3,
  "nome": "Terminal Rodoviário de Sobral 3",
  "cep": "70001-000",
  "rua": "Av. Central",
  "numero": "317",
  "bairro": "Centro",
  "id_cidade": 3
},
}
```

GET /rodoviar/rodoviaria/{id_rodoviaria} Get Rodoviaria

Parameters

Name

Description

id_rodoviaria required

integer

(path)

130

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8080/rodoviar/rodoviaria/130' \
  -H 'accept: application/json'
```

Request URL

http://localhost:8080/rodoviar/rodoviaria/130

Server response

Code

Details

200

Response body

```
{
  "id_rodoviaria": 130,
  "nome": "Terminal Secundário 0.2680629223894011",
  "cep": "60000-000",
  "rua": "Rua B",
  "numero": "12",
  "bairro": "Bairro",
  "id_cidade": 60
}
```

Download

Response headers

```
content-length: 149
content-type: application/json
date: Fri, 03 Jul 2026 00:59:10 GMT
```

6. Integração com a Base de Dados

Conexão ao Banco de Dados

- A conexão com o banco de dados PostgreSQL é estabelecida de forma direta e sob demanda através da biblioteca `psycopg2`.
- **Abertura de Conexão:** No início de cada função/rota (como `criar_rodoviaria`, `listar_rodoviaras`), o código invoca a função `get_connection()` para abrir um canal com o banco e cria um cursor (`conn.cursor()`) para executar as instruções SQL.
- **Fechamento de Recursos:** A aplicação adota a prática de fechar o cursor e a conexão (`cur.close()` e `conn.close()`) no bloco `finally` ou logo após o término da consulta, garantindo que os recursos do SGBD sejam liberados imediatamente, evitando vazamento de conexões.
- **Controle de Transações:** Para operações que modificam dados (POST, DELETE e PATCH), o código utiliza `conn.commit()` para consolidar as alterações ou `conn.rollback()` dentro de blocos `try/except` para desfazer as operações caso ocorra algum erro, mantendo a integridade dos dados.

Consultas, Visões ou Stored Procedures

- A aplicação faz uso exclusivo de consultas SQL diretas e simples (CRUD básico), Stored Procedures ou junções (JOINS) complexas no lado do SGBD:
- **Consultas Diretas por ID/Chave Estrangeira:** Para operações como o cadastro (`INSERT INTO rodoviaria ...`) ou busca por ID (`SELECT ... FROM rodoviaria WHERE id_rodoviaria=%s`), o código envia comandos brutos ao banco de dados passando parâmetros de forma segura (`%s`).
- **Mapeamento em Código:** Na rota `GET /rodoviaras`, a aplicação executa um `SELECT` simples na tabela `rodoviaria` trazendo apenas os IDs das chaves estrangeiras (como `id_cidade`). A transformação e estruturação desses dados vindos do banco em objetos legíveis (JSON) são feitas diretamente no backend em Python.
- **Construção Dinâmica de Queries:** Na rota de atualização parcial (PATCH), a lógica de decidir quais campos alterar não está delegada a uma procedure no banco. O Python intercepta os campos enviados pelo cliente através do método `model_dump(exclude_unset=True)` do Pydantic, monta a string do comando `UPDATE` dinamicamente concatenando os campos modificados e a envia pronta para o PostgreSQL executar.

7. Anexos

📁 Entrega 4

- *etapa4.sql* → Comandos SQL das visões, criação de usuários e permissões
- *printscreen_app/* → Pasta com imagens do sistema funcionando
- *codigo_app/* → Código-fonte da aplicação (em Python, JS, etc.)